



A hybrid differential evolution algorithm for parallel machine scheduling of lace dyeing considering colour families, sequence-dependent setup and machine eligibility

Debiao Li, Jing Wang, Rui Qiang & Raymond Chiong

To cite this article: Debiao Li, Jing Wang, Rui Qiang & Raymond Chiong (2020): A hybrid differential evolution algorithm for parallel machine scheduling of lace dyeing considering colour families, sequence-dependent setup and machine eligibility, International Journal of Production Research, DOI: [10.1080/00207543.2020.1740341](https://doi.org/10.1080/00207543.2020.1740341)

To link to this article: <https://doi.org/10.1080/00207543.2020.1740341>



Published online: 25 Mar 2020.



Submit your article to this journal [↗](#)



View related articles [↗](#)



View Crossmark data [↗](#)

A hybrid differential evolution algorithm for parallel machine scheduling of lace dyeing considering colour families, sequence-dependent setup and machine eligibility

Debiao Li ^{a,b,*}, Jing Wang^a, Rui Qiang^a and Raymond Chiong ^{a,c}

^a*School of Economics and Management, Fuzhou University, Fuzhou, People's Republic of China;* ^b*Antai College of Economics and Management, Shanghai Jiao Tong University, Shanghai, People's Republic of China;* ^c*School of Electrical Engineering and Computing, The University of Newcastle, Callaghan, Australia*

(Received 6 February 2019; accepted 1 March 2020)

Dyeing is the most time and energy-consuming process in textile production. Motivated by a dyeing overdue problem in a lace textile factory, we study a parallel machine scheduling problem with different colour families, sequence-dependent setup times, and machine eligibility restriction. An integer programming model is formulated to minimise the total tardiness. Given that the dyeing optimisation problem is strongly *NP*-hard, a hybrid differential evolution (HDE) algorithm embedded with chaos theory and two local search algorithms is proposed to solve real-world instances from the textile factory. In our proposed algorithm, a special encoding and decoding scheme is designed to deal with the machine eligibility constraint, and chaos theory is adopted to determine the parameter settings of the underlying differential evolution (DE) algorithm. To speed up convergence and improve search exploitation, two local search algorithms inspired by two dominance properties are developed to determine the optimal job sequence for parallel machines, such that the decision of the entire problem is simplified to the assignment of jobs among the machines, and the computational time required is significantly reduced. Comprehensive experiments based on 36 synthetically generated small to large-scale problem instances and 20 real-world industrial data sets confirm the efficacy of our proposed HDE over other DE variants.

Keywords: textile dyeing optimisation; parallel machine scheduling with sequence-dependent setup times; hybrid differential evolution; chaos theory; dominance property

1. Introduction

With the increasing pressure of global competition, textile firms have to strengthen their competitive advantage in terms of cost, quality, lead time, flexibility, and environmental impact. Production in a textile factory typically consists of three sequential processes: weaving, dyeing, and sewing. Among these three, dyeing is more critical due to not only the technical complexity involved, but also energy consumption and water pollution issues, which would affect the entire production process (Huynh and Chien 2018). With the environmental regulations in place for the textile industry in China, an increasing number of clean technologies, such as water purification technologies, environment-friendly dyes, advanced automatic dyeing vessels (dyeing machines), and scheduling techniques, among others, have been utilised to improve the dyeing productivity with a lower environmental impact. From an operational perspective, however, tardiness is still a significant problem. Recently, advanced scheduling models and algorithms have been proposed to reduce water and energy consumption as well as tardiness (Jiang, Yuan, and Li Sun 2010; Zhang et al. 2017).

The textile production due date is usually determined by subtracting the required delivery time (measured in terms of land or sea transportation) from the order due date. If orders are over the production due date, they have to be shipped by air in order to meet the order due date, which increases the shipping cost substantially. In the worst case, the textile order is overdue and this incurs a penalty cost. According to previous research, dyeing is the most time-consuming procedure in textile production and can potentially cause major delays (Zhang et al. 2017). The dyeing process consists of five main steps: pre-treatment, dyeing, finishing, drying, and quality control (Huynh and Chien 2018). The processing time is dependent on both the colour category and depth of a job. For example, all ordinary colours (which always have a light colour depth) would need around 2 h to dye, whereas a single colour would require about 6 and 8 h, respectively, for light and dark colour depths.

Our work is motivated by an overdue dyeing problem in a lace textile factory located in Fuzhou, China. This factory is the second largest in the lace textile segment nationwide. To increase productivity, the factory has invested in around 70

*Corresponding author. Email: Qiangrui_fzu@163.com



Figure 1. Automatic dyeing vessels in the studied factory.

automatic dyeing vessels, some of which are shown in Figure 1. Since dye (colour) and water are automatically transferred to dyeing vessels through pipelines, the setup for dyeing mainly involves vessel cleaning, and the time consumed is dependent on the relationship between the colours in successive jobs. Longer vessel cleaning times result in higher water consumption and delayed job completion. This tardiness problem can be solved by devising an appropriate dyeing schedule, which minimises the setup time between jobs and thus reduces both water consumption and tardiness.

In this paper, we study the above problem as a parallel machine scheduling problem with sequence-dependent setup times in lace dyeing. The contributions of our work can be summarised as follows:

- To minimise total tardiness, an integer programming model is formulated for the parallel machine scheduling problem with different colour families, sequence-dependent setup times and a capacity constraint in the lace dyeing process. This model is validated and optimally solved using CVX, a Matlab-based modelling system for convex optimisation, based on 16 synthetic small-scale data instances.
- An efficient hybrid differential evolution (HDE) algorithm, which is composed of the differential evolution (DE) framework, is proposed. The DE framework consists of a self-adaptive mechanism and two local search algorithms inspired by two dominance properties.
- We have used 20 real-world industrial data sets and 20 synthetic large-scale data instances generated according to the characteristics of the industrial data sets for testing and evaluation. The performance of the proposed HDE algorithm is compared to three DE variants.

In the next section, relevant studies on the dyeing problem, parallel machine scheduling with sequence-dependent setup times and machine eligibility restrictions, and DE algorithms are discussed. We then describe the problem formulation in Section 3 and explain the proposed HDE algorithm in detail in Section 4. After that, experimental results are presented in Section 5, and a summary of this work is provided in Section 6.

2. Related work

2.1. Dyeing optimisation

Production planning and scheduling techniques have been applied to optimise the dyeing process with due date constraints over the past decades (Stinson and Smith 1982; Jiang, Yuan, and Li Sun 2010; Chen 2009; Zhou et al. 2017; Huynh and Chien 2018). In one of the earlier studies, Stinson and Smith (1982) proposed a heuristic procedure to solve real-world single machine dyeing problems, and proved that their heuristic procedure was more effective than a branch and bound algorithm. Chen (2009) studied a single machine dyeing problem to minimise the completion time subject to periodic maintenance and due dates.

Other researchers have considered parallel machine problems (Hsu et al. 2009; Jiang, Yuan, and Li Sun 2010; Zhou et al. 2017; Huynh and Chien 2018), taking the environmental impact into consideration. For example, Jiang, Yuan, and Li Sun (2010) proposed a genetic algorithm (GA) to reduce the dyeing time as well as water consumption by ordering colours into groups, while Zhou et al. (2017) proposed a dynamic GA to reduce fresh-water consumption and reduce wastewater discharge based on the dye colour and depth. Recently, Huynh and Chien (2018) developed a multi-subpopulation GA with heuristics embedded (MSGGA-H) to minimise the makespan of batch dyeing schedules. They applied a random heuristic to generate initial solutions that can enhance the search process. A batching heuristic and an extreme heuristic were then designed and embedded in the proposed MSGGA-H to improve its ability to overcome local optima.

Due date related objectives are the most common objectives in dyeing optimisation studies. To minimise total tardiness, Hsu et al. (2009) proposed a scheduling approach for yarn-dyed textile manufacturing with four characteristics: multi-stage production, sequence-dependent setup time, hierarchical product structure, and group-delivery. The scheduling problem was decomposed into a sequence of sub-problems, and each of them was solved by a GA. Iterative improvement schemes were applied until a qualified solution was obtained. Similarly, Zhang et al. (2017) formulated a bi-objective parallel batch-processing machine scheduling problem to minimise total tardiness of jobs and maximise the vessel utilisation rates. They used a multi-objective artificial bee colony algorithm with a specialised encoding scheme and a problem-specific initialisation technique to solve the problem efficiently.

From the above review, we see that there have been several dyeing optimisation related studies. However, there is room for further work, as many practical constraints, such as those related to colour families, sequence-dependent setup times, and machine eligibility restrictions, have not been simultaneously considered along this line of research.

2.2. Parallel machine scheduling with sequence-dependent setup times and machine eligibility constraints

To minimise the makespan, a parallel-dedicated machine scheduling problem with a single setup server was formulated by Huang, Cai, and Zhang (2010). A polynomial-time algorithm was proposed to solve a special case of this problem, and a hybrid GA was applied to tackle general cases. Hamzadayi and Yildiz (2017) studied a parallel identical machine scheduling problem with sequence-dependent setup times and a single setup server. They compared GA and simulated annealing-based approaches to common dispatching rules by applying them on large-scale instances. The proposed GA yielded the highest solution quality within reasonable computational time. An unrelated parallel machine scheduling problem with sequence-dependent setup times, resource constraints, different release dates, and machine eligibility and precedence constraints was studied by Afzalirad and Rezaeian (2016). Two meta-heuristic algorithms, the GA and Artificial Immune System (AIS), were utilised to find quasi-optimal solutions. Their results indicated that both algorithms are efficient and effective for small-scale instances, but the AIS outperforms the GA on large-scale instances.

To minimise both earliness and tardiness, a non-delayed relax-and-cut algorithm based on Lagrangian relaxation was proposed by Paula, Robson Mateus, and Ravetti (2010) for unrelated parallel machine scheduling with sequence-dependent setup times. They obtained optimal solutions for instances with 180 jobs and six machines within reasonable CPU time. A parallel identical-machine scheduling problem with sequence-dependent setup was formulated as a new network-based mixed-integer programming (MIP) model by Anderson et al. (2013). The proposed model was shown to be efficient for large multi-machine problems. Hung, Bao, and Cheng (2017) studied a parallel machine scheduling problem with sequence-dependent setup times and machine-job dependent processing rates. In their work, each job has a time window delimited by its release and due dates. They proposed and tested several MIP models and found a three-phase heuristic-initialised MIP model to be the most effective in minimising earliness and tardiness costs. Kim, Song, and Jeong (2020) proposed an identical parallel machine scheduling problem with job splitting and sequence-dependent setup times. They designed meta-heuristics with two encoding schemes and three decoding methods to find near-optimal solutions for the problem. A mathematical model was introduced by Berkhout et al. (2020) to solve real-life scheduling problems in a complex multi-stage machine environment with non-triangular sequence-dependent setup times and shifting production bottlenecks. Their primary goal was to minimise the tardiness of customer orders, and the secondary objective was to maximise the production capacity.

Machine eligibility has also been considered in parallel machine scheduling problems. To minimise the makespan for a batch scheduling problem on non-identical and parallel machines with non-identical job sizes, two heuristics based on the First-Fit-Decreasing rule and Max-Min Ant System were proposed by Jia, Li, and Leung (2015). Zhou et al. (2016) proposed an effective DE-based hybrid algorithm to solve large-scale instances of the uniform parallel batch scheduling problem with different machine capacities and arbitrary job sizes. Similarly, José, Goncalves Tavares, and Leung (2019) developed a simple and effective iterated greedy algorithm to minimise the total flow time of jobs while solving parallel batch machines with unequal job release times.

In this study, the dyeing optimisation problem is defined as a parallel machine scheduling problem with sequence-dependent setup times, taking machine eligibility into consideration. As can be seen from the above literature review, only a limited number of studies have considered the parallel machine scheduling problem with different colour families, sequence-dependent setup times and the machine eligibility constraint at the same time.

2.3. DE algorithms

The parallel machine scheduling problem with sequence-dependent setup times and machine eligibility restriction is strongly NP-hard (Naderi-Beni et al. 2014). Therefore, heuristics are commonly applied in the relevant literature to solve practical

instances of such problems. DE, proposed by Storn and Price (1997), is one of the most widely used evolutionary algorithms (Blum et al. 2012) for large-scale optimisation problems. However, the performance of classic DE is significantly affected by its control parameters (e.g. population size, scaling factor F , and crossover rate CR).

To avoid premature convergence, chaos theory was applied by different researchers to obtain self-adaptive parameter settings in DE (e.g. see Wang et al. 2009; Yuan et al. 2008; Lu et al. 2010; Banitalebi, Ismail Abd Aziz, and Abdul Aziz 2016). Based on chaos theory, the logistic chaotic map has been used to update the CR and F adaptively, and chaotic sequences have been used to help overcome local optima and improve global exploration.

Greedy local algorithms (Yuan and Xu 2013; Myszkowski et al. 2018) or problem-inspired algorithms (Qian et al. 2006, 2009; Wu and Che 2019) have also been embedded in DE to balance global exploration and local exploitation. For example, for a multi-objective job shop problem, an adaptive Meta-Lamarckian strategy was employed by Qian et al. (2006) to dynamically decide on the neighbourhood selection in each generation after the DE exploration. Qian et al. (2009) also proposed a problem-dependent local search method with DE for multi-objective no-wait flow shop scheduling. Wu and Che (2019) employed a memetic algorithm based on DE to optimise a bi-objective unrelated parallel machine scheduling problem by minimising both the makespan and total energy consumption.

Inspired by the studies reviewed above, we propose to embed chaos theory in DE and hybridise it with local search in this study. It is also worth pointing out that DE has not been previously applied to solve dyeing optimisation problems.

3. Problem description

3.1. Problem statement

The parallel dyeing vessel scheduling problem with different job families, sequence-dependent setup times and machine eligibility restriction is denoted as $Pm | fmls, s_{jk}, M_j | \sum T_j$, where job families refer to the colour families of lace textile, and machine eligibility refers to the capacity of the dyeing vessel. The objective is to minimise total tardiness of the dyeing jobs. Notations used for this problem are shown in Table 1.

When a job is received, it is characterised by four attributes: colour family (l_j), weight (w_j), dyeing process time (p_j), and dyeing production due date (d_j). In the factory being studied, 15 common colour families (l_j) are classified into four categories based on the complexity of the dyeing process in ascending order, namely ordinary, single, double, and triple. These colour categories determine the associated processing time, p_j , as shown in Table 2. The more complex the dyeing process, the larger the value of p_j . For example, the dyeing time for the ordinary colour category is 2 h, while the dyeing time for ‘Double’ is 16 h. In a ‘Pull’ production system, the dyeing production due date (d_j) is calculated by subtracting the required delivery time from the job due date; it is also defined as the target due date in this paper. Usually, the value of d_j increases with the complexity of the dyeing process. In other words, the more complex the dyeing process, the larger the values of p_j and d_j , and $p_i \leq p_j$ implies that $d_i \leq d_j$. Each job $j \in \{1, 2, \dots, n\}$ has a colour family (l_j), a weight (w_j), a dyeing process time (p_j), and a due date of dyeing (d_j). Each vessel $k \in \{1, 2, \dots, m\}$ can only process one job at a time without preemption, and each has a capacity limit, g_k . It is assumed that a job cannot be split, such that job j can be assigned

Table 1. A summary of notations used in the mathematical model.

Indexes and parameters	Description
i', i, j	Job index $i', i \in \{0, 1, 2, \dots, n\}, j \in \{1, 2, \dots, n\}, i' \neq i \neq j$
k	Machine index $k \in \{1, 2, \dots, m\}$
n	The number of jobs
m	The number of machines
c	The number of colour families
d_j	The dyeing production due date of job j
p_j	The processing time of job j
l_j	The colour family of job $j, l_j \in \{1, 2, \dots, c\}$
w_j	The weight of job j
$S_{l_i l_j}$	The sequence-dependent setup time when l_j is preceded by l_i
g_k	The capacity of machine k
T_j	The tardiness of job j
M	A large number
Decision variables:	
Z_{ijk}	Binary variable, $Z_{ijk} = 1$ if job j is immediately preceded by job i on machine k , otherwise 0
C_j	The completion time of job j

Table 2. Colour classification and processing time of the given industrial case.

l_j	Colour family	Colour category	Colour depth	p_j
1	Standard white	Ordinary	Light	2
2	Standard black	Ordinary	Light	2
3	Snow-white	Single	Light	6
4	Fleshcolour	Single	Light	6
5	Apricot	Single	Light	6
6	Pink	Single	Light	6
7	Grey	Single	Light	6
8	Blue	Single	Dark	8
9	Yellow	Single	Dark	8
10	Purple	Single	Dark	8
11	Green	Single	Dark	8
12	Red	Single	Dark	8
13	Black	Single	Dark	8
14	Two colour mix	Double	Dark	16
15	Three colour mix	Triple	Dark	24

to machine k only if $w_j \leq g_k$. For instance, job j with $w_j = 200$ cannot be processed by vessel k with $g_k = 100$. Jobs cannot be merged even if they belong to the same colour family.

Fifteen colour families are categorised into two colour depths, namely light and dark, as shown in Table 2. If two successive jobs i and j are from the same colour family ($l_i = l_j$), they have the same colour depth. Thus, there is no setup time involved. However, if $l_i \neq l_j$, setup time is required. Since the dye (colour) and water are automatically transferred to vessels through pipelines, the setup for dyeing between two successive jobs mainly involves cleaning the vessel. The cleaning effort required, rough or thorough cleaning, is determined by the colour depth (light or dark colour depth) of the previous job. Rough cleaning is required when the preceding job has a light colour depth, while thorough cleaning is needed when the preceding job has a dark colour depth. For example, when l_i is snow-white and l_j is blue, rough cleaning is needed. The time and water consumed by a thorough cleaning are triple those required for rough cleaning. Therefore, the cleaning effort is used to measure the setup time between two successive jobs in this paper.

3.2. Mathematical formulation

A deterministic integer programming model is established for problem $Pm | fmls, s_{jk}, M_j | \sum T_j$. It is assumed that all jobs are released at time zero, and no setup is required for the first job. The general mathematical model is formulated as follows:

$$\min \sum_{j=1}^n T_j. \quad (1)$$

Subject to:

$$T_j = \max\{C_j - d_j, 0\} \quad \forall j, \quad (2)$$

$$\sum_{k=1}^m \sum_{i=0}^n Z_{ijk} = 1 \quad \forall j, \quad (3)$$

$$\sum_{j=1}^n Z_{0jk} = 1 \quad \forall k, \quad (4)$$

$$\sum_{j=1}^n Z_{ijk} \leq \sum_{i'=0}^n Z_{i'ik}, \quad i \in \{1, 2, \dots, n\} \quad \forall k, \quad (5)$$

$$C_j \geq C_i - M \cdot (1 - Z_{ijk}) + p_j + S_{l_i l_j} \quad \forall i, j, k, \quad (6)$$

$$g_k \geq w_j \cdot \sum_{i=0}^n Z_{ijk} \quad \forall j, k, \quad (7)$$

$$Z_{ijk} \in \{0, 1\} \quad \forall i, j, k. \quad (8)$$

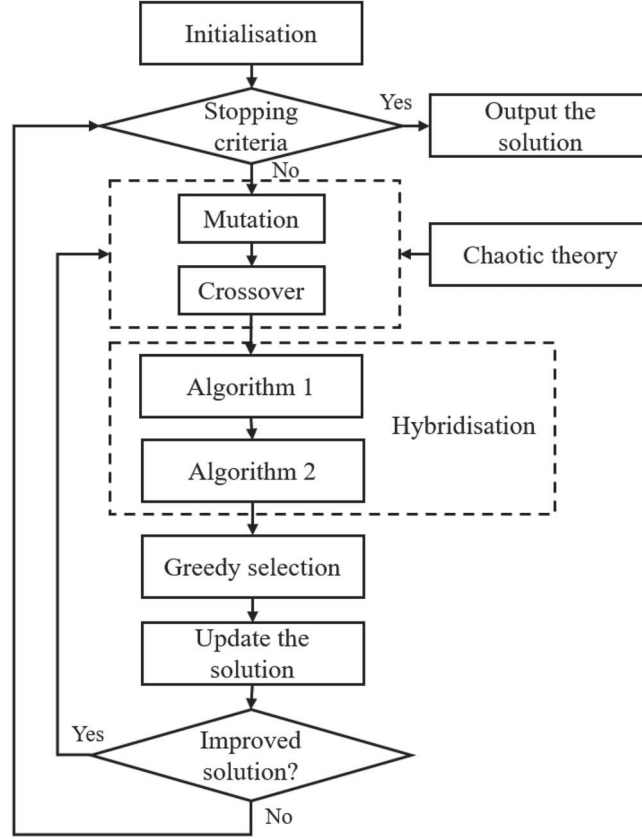


Figure 2. A flowchart of the HDE algorithm.

The objective function to minimise is the total tardiness, as shown in Equation (1). The tardiness of job j , T_j , is the maximum of the difference between job completion time C_j and dyeing due date d_j , and zero, as shown in Equation (2). Equation (3) ensures that job j can only be processed by one machine and each job in each machine can be processed only after the preceding job has been completed. A dummy job, Z_{0jk} , is introduced in Equation (4), which indicates that the dummy job 0 is placed at the beginning of each machine. Equation (5) ensures job j can be empty when job i is the last job on one machine for three consecutive jobs (i', i, j) . The completion time of job j , C_j , should be as large as the sum of the completion time of immediate preceding job i , C_i , the processing time, p_j , and the setup time, $S_{i|j}$, which is controlled by a large number, M , and Z_{ijk} , as shown in Equation (6). Equation (7) ensures that machine k can only process a job with weight w_j that does not exceed its capacity, g_k . The decision variable is binary, as stated in Equation (8).

4. Solution methods

Since the problem at hand is strongly *NP*-hard, a hybrid approach – HDE – embedded with chaos theory and two local search algorithms is proposed to efficiently minimise the total tardiness. A flowchart of the HDE algorithm can be seen in Figure 2. In the beginning, feasible solutions are initialised by a special encoding and decoding scheme, as described in Section 4.1. Then, a self-adaptive differential evolution (SDE) algorithm based on the classic DE algorithm and chaos theory is developed, details of which are presented in Sections 4.2 and 4.3, respectively. Finally, two local search algorithms based on dominance properties are proposed to speed up convergence and improve the local search, as shown in Section 4.4.

4.1. Initialisation

Given the pre-defined population size, q , the population is denoted by a $q \times n$ matrix $\mathbf{A}^t$, where t is the number of generations. The entry in row e and column j is denoted by a'_{ej} , where $e \in (1, 2, \dots, q)$. Matrix $\mathbf{A}^t$ is also represented by its rows $\mathbf{a}'_e$, then $\mathbf{A}^t = [\mathbf{a}'_1, \mathbf{a}'_2, \dots, \mathbf{a}'_q]^T$. Each row $\mathbf{a}'_e$ consists of n real numbers, and is decoded as the scheduling solution, Ψ_k . When job j is received, it can be assigned to machine k only if $w_j \leq g_k$. Machines that are eligible for job j based on these capacity

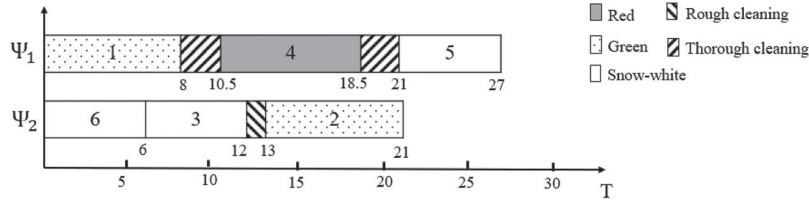


Figure 3. A Gantt chart of the schedule decoded from the given solution.

constraints are denoted as set Ω_j . To ensure the feasibility of initial solutions $\mathbf{A}^1$, an encoding scheme based on the random key representation (Zhang et al. 2017) is applied.

$$a_{ej}^t \in \cup_{k \in \Omega_j} (k - 1, k]. \quad (9)$$

An illustrative example is presented below, where there are six jobs ($n = 6$) processed in two dyeing vessels ($m = 2$), with capacity constraints $g_1 = 15$, $g_2 = 30$, the processing times $p_j = [8, 8, 6, 8, 6, 6]$, weights $w_j = [9, 17, 24, 10, 11, 22]$, due dates $d_j = [21, 24, 8, 29, 15, 20]$, and colour families $l_j = [11, 11, 3, 12, 3, 3]$. The initial population, $\mathbf{A}^1$, is shown in Matrix 10. Real numbers in the matrix are rounded to four decimal places, and the population size (q) is set to be 5 for illustration purposes. For example, because $w_1 \leq g_1$, $w_2 \geq g_1$, so $a_{e1}^1 \in (0, 2]$ and $a_{e2}^1 \in (1, 2]$, respectively.

$$\mathbf{A}^1 = \begin{pmatrix} 0.0782 & 1.6781 & 1.6504 & 0.4842 & 0.9357 & 1.0012 \\ 0.3586 & 1.9221 & 1.1143 & 0.9771 & 1.1625 & 1.9021 \\ 0.1143 & 1.8399 & 1.9687 & 0.3674 & 0.2276 & 1.8996 \\ 0.9862 & 1.0634 & 1.3642 & 1.9687 & 0.1883 & 1.9083 \\ 1.7129 & 1.2264 & 1.1436 & 0.9426 & 1.7823 & 1.9811 \end{pmatrix}. \quad (10)$$

During the decoding process, the machine assignment of job j is indicated by the ceiling value of a_{ej}^t , which is denoted as $\lceil a_{ej}^t \rceil$. On machine k , the job sequence is determined by the decimal part of a_{ej}^t in ascending order, which can be represented by $a_{ej}^t - \lfloor a_{ej}^t \rfloor$. For example, the ceiling value of the first row $\mathbf{a}_1^1 = [0.0782, 1.6781, 1.6504, 0.4842, 0.9357, 1.0012]$ of $\mathbf{A}^1$ is calculated, such that $a_{1j}^1 = \lceil a_{1j}^1 \rceil = [1, 2, 2, 1, 1, 2]$. Therefore, jobs 1, 4 and 5 are assigned to machine 1, and jobs 2, 3 and 6 are assigned to machine 2. Then, job sequences for machines 1 and 2 are obtained by the decimal parts of a_{1j}^1 , calculated by $a_{1j}^1 - \lfloor a_{1j}^1 \rfloor$. Since $a_{11}^1 - \lfloor a_{11}^1 \rfloor < a_{14}^1 - \lfloor a_{14}^1 \rfloor < a_{15}^1 - \lfloor a_{15}^1 \rfloor$, the job sequence for machine 1 is $\Psi_1 = (1, 4, 5)$. Job sequence $\Psi_2 = (6, 3, 2)$ can be obtained similarly. The corresponding Gantt chart of this solution with the total tardiness time (16) is shown in Figure 3.

4.2. HDE

The proposed HDE is based on the classic DE (Storn and Price 1997), which is a very simple but effective algorithm that works by generating trial parameter vectors. Due to its simplicity and effectiveness, DE has been successfully applied to solve a wide range of optimisation problems. In DE, a new population $\mathbf{A}^{t+1}$ is generated using mutation and crossover operators. These mutation and crossover operators are employed to generate new candidate vectors (target vectors), and a greedy selection scheme is applied to determine the surviving vectors for the next generation. The above process is repeated until the termination criterion is met.

4.2.1. Mutation

The commonly used mutation operator, 'DE/rand/1', is applied to yield the mutant vector $\mathbf{V}_e^{t+1}$, $\mathbf{V}_e^t = [v_{e1}^t, v_{e2}^t, \dots, v_{en}^t]$. For each target vector $\mathbf{a}_e^t$, $\mathbf{a}_e^t = [a_{e1}^t, a_{e2}^t, \dots, a_{en}^t]$. The generation of mutant vector $\mathbf{V}_e^{t+1}$ is based on scaling factor F and three different randomly selected target vectors $\mathbf{a}_{r_1}^t$, $\mathbf{a}_{r_2}^t$, and $\mathbf{a}_{r_3}^t$, as described in Equation (11) below.

$$\mathbf{V}_e^{t+1} = \mathbf{a}_{r_1}^t + F \times (\mathbf{a}_{r_2}^t - \mathbf{a}_{r_3}^t), \quad r_1 \neq r_2 \neq r_3 \neq e. \quad (11)$$

The scaling factor, F , controls the amplification of differential variation $\mathbf{a}_{r_2}^t - \mathbf{a}_{r_3}^t$, where $F \in (0, 2)$ (Storn and Price 1997).

However, the mutation operator may generate vectors that fail to meet the capacity constraints, necessitating a repair modification. In the repair modification, only elements that violate the capacity constraints are re-generated. For

	Job 1	Job 2	Job 3	Job 4	Job 5	Job 6	
$\mathbf{a}_2^1$	0.3586	1.9221	1.1143	0.9771	1.1625	1.0921	Mutation
$\mathbf{a}_3^1$	0.1143	1.8399	1.9687	0.3674	0.2276	1.8996	
$\mathbf{a}_4^1$	0.9826	1.0634	1.3642	1.9687	0.1883	1.9083	
F=0.4							
$\mathbf{V}_1^2$	0.0098	2.2327	1.3561	0.3366	1.1782	1.8986	Replace
↓ Modified							
$\mathbf{V}_1^{2'}$	0.0098	1.9219	1.3561	0.3366	1.1782	1.8986	

Figure 4. An example of mutation and modification.

	Job 1	Job 2	Job 3	Job 4	Job 5	Job 6
$\mathbf{V}_1^2$	0.0098	1.9219	1.3561	0.3366	0.1782	1.8986
$\downarrow r(1) \leq CR$ $\downarrow rnb(e) = 2$ $\downarrow r(4) \leq CR$ $\downarrow r(5) \leq CR$						
$\mathbf{U}_1^2$	0.0098	1.9219	1.6504	0.3366	0.1782	1.0012
$\uparrow r(3) > CR$ $\uparrow r(6) > CR$						
$\mathbf{a}_1^1$	0.0782	1.6781	1.6504	0.4842	0.9359	1.0012

Figure 5. An example of crossover.

example, a mutation operator and a repair modification of the given example in matrix $\mathbf{A}^1$ are shown in Figure 4. Let $e = 1$, $r_1 = 2$, $r_2 = 3$, $r_3 = 4$, and $F = 0.4$, the mutant vector $\mathbf{V}_1^2$ can be obtained via Equation (11), i.e. $\mathbf{V}_1^2 = [0.0098, 2.2327, 1.3561, 0.3366, 1.1782, 1.8986]$. Since $w_2 > g_1$, $v_{e2}^1 \in (1, 2]$. However, $v_{12}^2 = 2.2327$, therefore, v_{12}^2 is infeasible and needs to be repaired. A new $v_{12}^{2'}(1.9219)$ is randomly chosen from $(1, 2]$ based on Equation (9), which is used to replace v_{12}^2 . Since the remaining jobs meet the capacity constraints defined by Equation (7), $\mathbf{V}_1^{2'} = [0.0098, 1.9219, 1.3561, 0.3366, 1.1782, 1.8986]$.

4.2.2. Crossover

After the mutation phase, a crossover operator is applied to obtain a trial vector $\mathbf{U}_e^{t+1} = [u_{e1}^{t+1}, u_{e2}^{t+1}, \dots, u_{en}^{t+1}]$, which is controlled by the crossover rate, CR . Here, the crossover rate is pre-determined, with $CR \in [0, 1]$. The trail vector, $\mathbf{U}_e^{t+1}$, is formed from elements selected from mutant vector $\mathbf{V}_e^{t+1}$ and target vector $\mathbf{a}_e^t$.

The trial element, u_{ej}^{t+1} , is generated by Equation (12) below.

$$u_{ej}^{t+1} = \begin{cases} v_{ej}^{t+1} & \text{if } r(j) \leq CR \text{ or } j = rnb(e), \\ a_{ej}^t & \text{otherwise,} \end{cases} \quad (12)$$

where $r(j)$ is a random number between $(0, 1)$, and $rnb(e)$ is a randomly chosen index from $\{1, 2, \dots, n\}$, which ensures that $\mathbf{U}_e^{t+1}$ gets at least one element from $\mathbf{V}_e^{t+1}$. For the example from Section 4.2.1, let $CR = 0.9$, $rnb(e) = 2$, and $r(1) = 0.7881$, $r(2) = 0.8913$, $r(3) = 0.9821$, $r(4) = 0.7812$, $r(5) = 0.1145$, and $r(6) = 0.9001$. The trial element, u_{1j}^2 , is obtained by Equation (12), then $\mathbf{U}_1^2 = [0.0098, 1.9219, 1.6504, 0.3366, 1.1782, 1.0012]$, as shown in Figure 5.

4.2.3. Greedy selection

After the crossover phase, a selection process is performed to obtain an updated target vector $\mathbf{a}_e^{t+1} = [a_{e1}^{t+1}, a_{e2}^{t+1}, \dots, a_{en}^{t+1}]$. The target vector, $\mathbf{a}_e^t$, is compared to its corresponding trial vector $\mathbf{U}_e^{t+1}$ using the one-to-one greedy criterion. When the objective value of the trial vector $\mathbf{U}_e^{t+1}$ is no worse than the objective value of the target vector $\mathbf{a}_e^t$, then $\mathbf{a}_e^{t+1}$ is updated by $\mathbf{U}_e^{t+1}$. Given our aim to minimise the objective function, Equation (13) is applied for the selection. If the objective value of $\mathbf{U}_e^{t+1}$ is no larger than $\mathbf{a}_e^t$, then $\mathbf{a}_e^{t+1} = \mathbf{U}_e^{t+1}$. Otherwise, $\mathbf{a}_e^t$ is retained.

$$\mathbf{a}_e^{t+1} = \begin{cases} \mathbf{U}_e^{t+1} & \text{if } f(\mathbf{U}_e^{t+1}) \leq f(\mathbf{a}_e^t), \\ \mathbf{a}_e^t & \text{otherwise.} \end{cases} \quad (13)$$

4.3. Self-adaption via chaos theory

The DE parameters (including F and CR) are constant in basic DE. However, they play a key role in the performance of DE. Therefore, a self-adaptive mechanism by chaos theory can be embedded in DE (Lu et al. 2010). Chaos often exists in nonlinear systems, which is the highly unstable motion of a deterministic system in finite phase space. Chaotic sequences show unpredictable long-term behaviour due to their sensitivity to the initial state conditions. Because chaotic optimisation methods have the characteristics of ergodicity, stochasticity, and irregularity, they are always able to increase the algorithm's exploitation capability in the search space and enhance its convergence.

In this work, the logistic map is applied as the self-adaptive mechanism of F and CR in DE, which is one of the simplest dynamic systems exhibiting chaotic behaviour (Yuan et al. 2008). F and CR are updated based on the logistic map, as shown in Equation (14).

$$y^{t+1} = \mu \times y^t \times (1 - y^t), \quad (14)$$

where y^t is the chaotic number, which can be F or CR . μ is a control parameter, $0 \leq \mu \leq 4$. A deterministic chaotic dynamic number is created by Equation (14) in the range $(0, 1)$, $y^1 \in (0, 1)$, and when $\mu = 4$, $y^0 \notin \{0, 0.25, 0.5, 0.75, 1\}$. During the evolutionary process, F and CR are properly initialised first, and then dynamically adapted for each iteration. This self-adaptive mechanism by chaos theory in DE not only weakens the influence of initial values on the performance of the algorithm, but also improves the global convergence and ability to escape from local optima.

4.4. Local search algorithms

In this study, the proposed HDE algorithm is composed of the evolutionary framework of DE and a set of local search algorithms inspired by two dominance properties. We use Algorithm 1 to minimise the sequence-dependent setup time based on the property of the setup effort associated with the colour family. We then use Algorithm 2 to minimise the tardiness according to the relationship of the processing time and the due date.

Property 1 Assume that $p_i \leq p_j$, $d_i \leq d_j$, then jobs with the same colour should be processed sequentially on one machine, otherwise job j with dark colour depth should be preceded by job i with light colour depth.

Proof It is intuitive that jobs with the same colour should be processed sequentially on one machine, such that their setup times can be minimised. When job j and job i are of different colours, however, job j with dark colour depth should be preceded by job i with light colour depth. The detailed proof is shown as follows. It is assumed that the more complex the dyeing process is, the larger p_j and d_j are. The colour family of job j , l_j , is sorted in ascending order. Therefore, $l_i \leq l_j$ implies that $p_i \leq p_j$, and $p_i \leq p_j$ implies that $d_i \leq d_j$. If $l_i \leq l_j$, then $S_{l_i l_i} \leq S_{l_i l_j}$. Consider a schedule in which job j is preceded by job i . Let $C_{i'}$ denote the completion time of job i' , where job i is immediately preceded by job i' on machine k , and C_1 and C_2 denote the completion times of job i and job j , respectively. We thus have $C_1 = C_{i'} + S_{l_i l_i} + p_i$, $C_2 = C_1 + S_{l_i l_j} + p_j$. Then, exchange job i and job j , and the new completion times of job j and job i are denoted as C'_1 and C'_2 , respectively. We now have $C'_1 = C_{i'} + S_{l_i l_j} + p_j$, $C'_2 = C'_1 + S_{l_j l_i} + p_i$. Since $S_{l_i l_i} \leq S_{l_i l_j}$, $S_{l_i l_j} \leq S_{l_j l_i}$ and $p_i \leq p_j$, we have $C'_1 \geq C_1$, $C'_2 \geq C_2$. ■

Let T_i , T'_i and T_j , T'_j denote the tardiness of job i and job j before and after the job exchange, respectively.

$$T'_j + T'_i - (T_i + T_j) = (C'_1 - d_j)^+ + (C'_2 - d_i)^+ - ((C_1 - d_i)^+ + (C_2 - d_j)^+) \quad (15)$$

because $C'_1 \geq C_1$, $C'_2 \geq C_2$, then

$$T'_j + T'_i - (T_i + T_j) \geq (C_1 - d_j)^+ + (C_2 - d_i)^+ - ((C_1 - d_i)^+ + (C_2 - d_j)^+). \quad (16)$$

Moreover, $d_i \leq d_j$, $C_1 \leq C_2$, $C_1 - d_i - (C_1 - d_j) = C_2 - d_i - (C_2 - d_j) = d_j - d_i \geq 0$, and the non-negative increasing function x^+ is convex, we obtain

$$\begin{aligned} T'_j + T'_i - (T_i + T_j) &\geq (C_1 - d_j)^+ + (C_2 - d_i)^+ - ((C_1 - d_i)^+ + (C_2 - d_j)^+) \\ &= ((C_2 - d_i)^+ - (C_2 - d_j)^+) - ((C_1 - d_i)^+ - (C_1 - d_j)^+) \geq 0. \end{aligned} \quad (17)$$

Therefore, the total tardiness does not decrease after the interchange between jobs i and j , which indicates that job j with dark colour depth should be preceded by job i with light colour depth.

From Table 2, we see that there are 15 colour families categorised into two colour depths, namely light and dark. They are also classified into four categories based on the complexity of the dyeing process in ascending order, which determines the associated processing time p_j . The more complex the dyeing process, the larger the value of p_j , and $p_i \leq p_j$ implies that $d_i \leq d_j$. If two consecutive jobs i and j are from the same colour family ($l_i = l_j$), there is no setup time, since they are in the same colour depth category. Therefore, jobs with the same colour should be processed consecutively on one machine. However, if $l_i \neq l_j$, setup time is required. The time and water consumed by a thorough cleaning are triple those required for rough cleaning. To reduce setup time, jobs with dark colour depth should be preceded by jobs with light colour depth. Therefore, jobs should be sorted by their colour family and colour depth, as shown in Algorithm 1.

Algorithm 1

Input: Schedule Ψ_k ; colour family l_j ;

Output: Schedule Ψ'_k ;

```

1: for  $k = 1 \rightarrow m$  do
2:   if  $\Psi_k \neq \emptyset$  then
3:     obtain  $\Psi'_k$  by sorting  $l_j$  in ascending order
4:   end if
5: end for
6: Return  $\Psi'_k$ .

```

Property 2 Assume that $p_i \leq p_j$, $d_i \leq d_j$, then there exists an optimal sequence in which job j is preceded by job i within the same colour family.

Proof The machine eligibility constraint is considered beforehand when selecting a machine, and sequence-dependent setup times are already adjusted by Property 1. Moreover, Property 2 holds only when job i and job j belong to the same colour family. Therefore, the proof of Property 2 is the same as Leung, Kelly, and Anderson (2004), despite the machine eligibility constraint and sequence-dependent setup times being considered. ■

Algorithm 2 minimises tardiness by further improving Ψ'_k based on Property 2. First, Ψ'_k is updated by Algorithm 1, such that jobs with the same colour are processed consecutively on one machine, and jobs with dark colour depth are preceded by jobs with light colour depth. All jobs in Ψ'_k are then scheduled in ascending order of colour family l_j . This means p_j and d_j are already sorted in ascending order of l_j by Algorithm 1. However, for jobs in the same colour family, d_j may not follow Property 2. Since p_j is identical for jobs in the same colour family, Ψ'_k is further updated by sorting d_j in ascending order for jobs in the same colour family, as shown in Algorithm 2. Therefore, Property 2 holds when job i and job j belong to the same colour family for schedules following Property 1.

After the crossover phase, trial vector $\mathbf{U}_e^{t+1}$ is decoded as its corresponding schedules, and then the schedules are updated by local search algorithms. For example, trial vector $\mathbf{U}_1^2$ is decoded as $\Psi_1 = [1, 5, 4]$ and $\Psi_2 = [6, 3, 2]$, which are shown in Figure 6(a); with $l_1 = 11$, $l_4 = 12$, $l_5 = 3$ in Ψ_1 , and $l_2 = 11$, $l_3 = 3$, $l_6 = 3$ in Ψ_2 , the total tardiness is 5.4, which is minimised by Algorithm 1. Since $l_5 < l_1 < l_4$, $l_6 = l_3 < l_2$, $\Psi'_1 = [5, 1, 4]$, and the total tardiness becomes 4, as shown in Figure 6(b). Later, tardiness is further minimised by Algorithm 2. Since $l_6 = l_3$ and $d_3 < d_6$, $\Psi''_2 = [3, 6, 2]$, the total tardiness becomes 0, as shown in Figure 6(c).

Note that the sequence of trail element u_{ij}^{t+1} is updated according to the job sequence change in Ψ'_k . For the same example, $u_{ij}^2 = [0.0098, 1.9219, 1.6504, 0.3366, 0.1782, 1.0012]$, then $u_{ij}^{2'}$ and $u_{ij}^{2''}$ are updated based on Ψ' and Ψ'' , such that $u_{ij}^{2'} = [0.1782, 1.9219, 1.6504, 0.3366, 0.0098, 1.0012]$ and $u_{ij}^{2''} = [0.1782, 1.9219, 1.0012, 0.3366, 0.0098, 1.6504]$.

Algorithm 2**Input:** Schedule Ψ'_k ; due date d_j ; colour family l_j ;**Output:** Schedule Ψ''_k ;

```

1: for  $k = 1 \rightarrow m$  do
2:    $\gamma = |l_j|, j \in \Psi'_k$ 
3:    $\Psi'_k = [\psi'_1, \psi'_2, \dots, \psi'_\gamma]$ 
4:   for  $b = 1 \rightarrow \gamma$  do
5:     obtain  $\psi''_b$  by sorting  $d_j$  in ascending order
6:   end for
7:    $\Psi''_k = [\psi''_1, \psi''_2, \dots, \psi''_\gamma]$ 
8: end for
9: Return  $\Psi''_k$ .

```

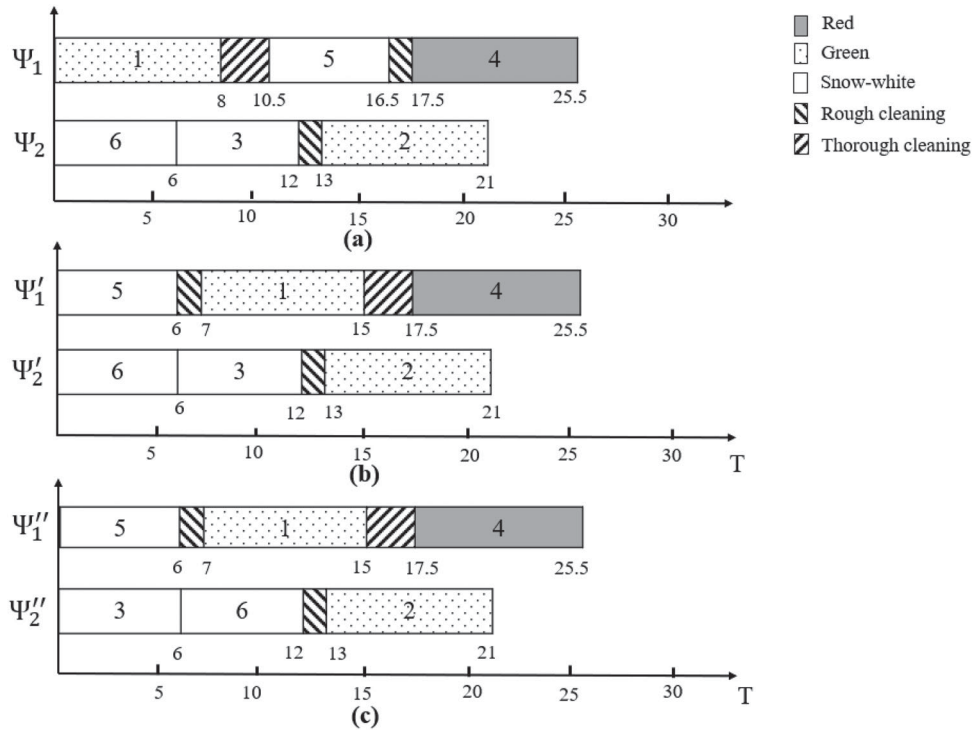


Figure 6. A Gantt chart of the schedule's updated solutions using Algorithms 1 and 2.

In this case, the sequences of all jobs are already determined beforehand, and the decision of the whole problem is reduced to an assignment problem. This is done by adding Equations (18) and (19) to the general mathematical model, which consists of Equations (1)–(8), to speed up the computation.

$$S_{l_j l_i} \geq S_{l_i l_j} - M \cdot (1 - Z_{ijk}) \quad \forall i, j, k, \quad (18)$$

$$d_j \geq d_i - M \cdot (1 - Z_{ijk}) \quad \forall i, j, k. \quad (19)$$

Here, Equations (18) and (19) ensure that if job j is immediately preceded by job i on machine k , then $S_{l_j l_i}$, d_j should be greater than or equal to $S_{l_i l_j}$, d_i , respectively, controlled by a large number, M , and Z_{ijk} .

5. Experimental results and analysis

To evaluate the performance of the proposed algorithm, computational experiments were conducted using both industrial and synthetically generated data sets. The industrial data sets were obtained from a lace textile factory in Fuzhou, China, while the synthetic data sets were generated based on empirical distributions of the industrial data. Three different parameter

Table 3. Specifications of the dyeing vessels.

Designed capacity(kg)	Practical capacity	Quantity
4	4	16
5	5	1
25	15	1
50	35	7
75	70	10
150	100	9
250	180	6
400	320	3
500	420	10
750	650	4
1000	800	1

Table 4. The colour family distribution.

l_j	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Percentage	29	11	5.5	4	2.5	2	5.5	4.5	3	5.5	3	7.5	12	4.5	0.5
p_j	2	2	6	6	6	6	6	8	8	8	8	8	8	16	24
d_j	8	8	24	24	24	24	24	24	24	24	24	24	24	24	24

Table 5. The interval distribution of weights.

Interval	[0.5,100]	(100,200]	(200,300]	(300,400]	(400,500]	(500,600]	(600,700]	(700,800]
Percentage	66.5	16.5	5	5	2	2	2	1

settings were tested: the number of jobs (n), the number of machines (m), and the number of colour families (c). Our objective was to minimise the total tardiness of all jobs. The average and best objective values obtained by our proposed HDE algorithm from 10 independent runs were compared with those obtained by the classic DE, SDE, and SDE with Algorithm 1 based on the colour family property (SDEC). In the synthetic sample tests, solutions obtained by CVX on small-scale data sets were also compared with these four DE-based algorithms, whereas solutions obtained by DE-based algorithms were compared with the industrial solutions for industrial sample tests. The tardiness was measured in hours, and the computational time (CT) was measured in seconds. The algorithms were implemented in a MATLAB R2016a environment and the experiments were conducted on a desktop computer with an Intel i5-8250 CPU (1.8GHz) and 8-GB RAM.

5.1. Test data sets

To verify the efficacy of the proposed algorithm, both industrial and synthetic data sets were tested. The characteristics of industrial data sets are presented in Section 5.1.1, and synthetic data sets, generated randomly based on empirical distributions of the industrial data sets, are described in Section 5.1.2.

5.1.1. Industrial data sets

In the factory being studied, there are 68 dyeing vessels. Each type of dyeing vessel has its own capacity and quantity, as summarised in Table 3. The practical capacity of a dyeing vessel is usually about 80% of its designed capacity, and the weight of the job cannot exceed the practical capacity. Based on practical experience, rough cleaning takes about an hour, while thorough cleaning takes around 2.5 h.

The characteristics of 3520 orders from 20 typical workdays were analysed, which include the colour family and weight. The colour family determines the processing time and dyeing production due date, and the empirical distribution is shown in Table 4 as well as Figure 7(a). The weight determines the eligibility of the vessel for the job, and the interval distribution is shown in Table 5 as well as Figure 7(b).

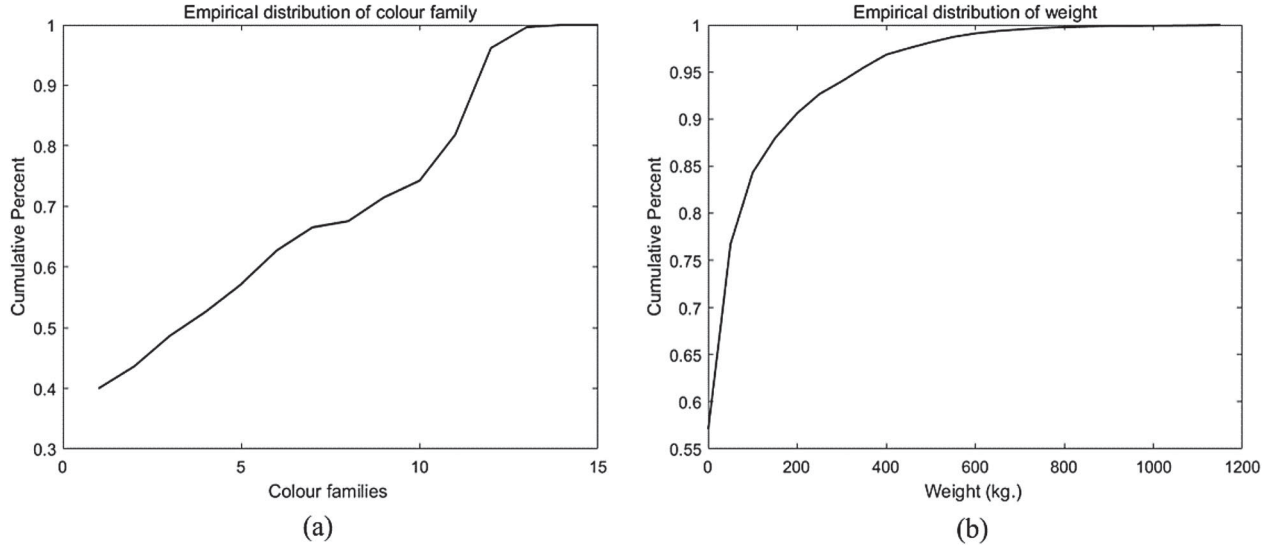


Figure 7. Empirical distributions of weights and colour families of industrial orders.

5.1.2. Synthetic data sets

Synthetic data sets were randomly generated based on the empirical distributions of industrial data sets. The colour families of these artificial jobs were generated based on the empirical distribution, as shown in Table 4. The processing time is the same as industrial data sets. Usually, if $p_i \leq p_j$, then $d_i \leq d_j$ in the case we studied. To test the effect of due dates, a modified (T, RDD) rule (Wan and Yen 2002) was used to generate the due dates, where T is the tardiness factor, and RDD is the relative range of the due windows. In m parallel machines, due dates of jobs were randomly selected from the due windows $[(1 - T - RDD/2) * MS/m, (1 - T + RDD/2) * MS/m]$, where $T = 0.3$, $RDD = 1$, and MS is the total processing time of all jobs.

5.2. Parameters of algorithms

The performance of DE is very sensitive to its parameters, such as F , CR , population size q , and the stopping criterion. In this study, we set the initial scaling factor $F^1 = 0.4$, initial crossover rate $CR^1 = 0.9$, and control parameter $\mu = 4$ (Yuan et al. 2008). Let $q = n \times m$, where n is the number of jobs and m is the number of machines. To reduce computational time, it was set to be 3000 when $q > 3000$. The stopping criterion was based on n non-improving iterations.

5.3. Experiments on synthetic problem instances

Both small-scale and large-scale synthetic data sets were used for this series of experiments. The data scale, which gradually increases, is denoted as $n \times m \times c$.

For small-scale data instances, the CVX solver was applied to obtain optimal solutions by solving the integer programming model formulated in Section 3. As shown in Table 6, optimal solutions obtained by CVX on the 16 small-scale synthetic data instances are compared with those of DE, SDE, SDEC, and HDE. Since all DE variants managed to solve the instances within 3 s, the computational time of DE variants is not reported. When $n = 12$, $m = 4$, and $c = 4$, optimal solutions for the general mathematical model presented in Section 3.2 and the updated model presented in Section 4.4 cannot be found by CVX in two hours. In Table 6, we denote the computational times for solving the general and updated mathematical models as CT_1 and CT_2 , respectively. From the results, it is clear that the computational time is greatly reduced when Equations (18) and (19) are added to the general mathematical model. For DE and its variants, we see that DE itself yields optimal values for 14 out of the 16 samples, whereas SDE, SDEC, and HDE obtain optimal values for 15 of them. Compared with DE, SDE, and SDEC, the proposed HDE algorithm reduces total tardiness by 7.05%, 2.03%, and 1.98%, respectively, on average.

Since the problem is strongly NP-hard, CVX can only be applied to solve considerably small-scale instances. For large-scale data sets, the proposed HDE algorithm is compared with the other three DE variants: DE, SDE, and SDEC, and the results are shown in Table 7. Compared to DE, SDE, and SDEC, the proposed HDE algorithm reduces total tardiness by

Table 6. Computational results on small-scale synthetic data sets.

No.	Problem scale $n \times m \times c$	CVX			Best objective value				Average objective value			
		Optimal	CT_1	CT_2	DE	SDE	SDEC	HDE	DE	SDE	SDEC	HDE
1	$4 \times 2 \times 2$	4	1.4	2.9	4*	4*	4*	4*	4.2	4.1	4.5	4
2	$6 \times 2 \times 2$	3	1.8	3.5	3*	3*	3*	3*	3.1	3	3	3
3	$8 \times 2 \times 3$	2	2.4	5.0	2*	2*	2*	2*	2	2	2	2
4	$9 \times 2 \times 3$	7	17.1	5.1	7*	7*	7*	7*	7.6	7	7	7
5	$6 \times 3 \times 2$	4	5.0	3.6	4*	4*	4*	4*	4	4	4	4
6	$8 \times 3 \times 2$	4	11.1	4.9	4*	4*	4*	4*	4.1	4	4	4
7	$8 \times 4 \times 2$	5	8.3	5.3	5*	5*	5*	5*	5	5	5	5
8	$8 \times 4 \times 3$	7	7.8	5.5	7*	7*	7*	7*	7	7	7	7
9	$9 \times 3 \times 3$	8	8.8	4.8	9	8*	8*	8*	9.5	9.1	8.75	9.15
10	$9 \times 4 \times 3$	17	40.4	6.4	17*	17*	17*	17*	17	17	17	17
11	$10 \times 2 \times 4$	6.5	193.4	4.7	6.5*	6.5*	6.5*	6.5*	6.5	6.5	6.5	6.5
12	$10 \times 3 \times 3$	3	20.4	5.7	3*	3*	3*	3*	3	3	3	3
13	$10 \times 4 \times 3$	8	193.4	7.3	8*	8*	8*	8*	8.1	8	8	8
14	$11 \times 3 \times 4$	19	1581.6	5.9	19*	19*	19*	19*	19	19	19	19
15	$11 \times 4 \times 4$	5	839.7	7.5	5*	5*	5*	5*	5.4	5	5	5
16	$12 \times 4 \times 4$	5	—	8.7	9	5*	5*	5*	11.6	7.4	7.3	5.2

*Indicates the optimal value.

Table 7. Computational results on large-scale synthetic data sets.

No.	Problem scale $n \times m \times c$	Average objective value				Average computational time				Standard deviation			
		DE	SDE	SDEC	HDE	DE	SDE	SDEC	HDE	DE	SDE	SDEC	HDE
1	$20 \times 5 \times 4$	32.6	16.1	12.3	8.3	3	3	3	3.3	10.3	4.3	3.4	2.1
2	$25 \times 5 \times 4$	9.4	7.2	7	7	3	3	4	3.8	2.7	0.6	0	0
3	$30 \times 5 \times 4$	86.5	21.8	16.3	14	4	6	7	7.8	10	11.3	5.3	1.5
4	$35 \times 5 \times 4$	94.5	19.9	15.4	17.6	7	11	11	7.1	14	6.8	5.5	2.9
5	$40 \times 5 \times 4$	196.1	31.9	15	5.1	5	11	15	11.2	14.8	21	10	5.2
6	$50 \times 5 \times 5$	402.2	34.6	33.1	30.6	15	29	35	36	37.2	3.3	2.9	2.5
7	$55 \times 5 \times 5$	214.7	12.9	2.1	0	14	21	22	14.9	30.2	14	4.3	0
8	$60 \times 10 \times 5$	324.9	76.9	63	36.3	110	219	259	171.9	25	27.9	18.3	1
9	$70 \times 10 \times 5$	406.9	48.5	37.8	19.5	82	253	325	281.1	40.5	13.6	12	4.9
10	$75 \times 10 \times 5$	548.3	80.7	55.7	37.9	87	286	287	336.8	45.4	51.9	15.2	3.1
11	$80 \times 10 \times 5$	359.2	75.1	63.2	43.1	132	322	354	361	28.1	32	13.3	10.9
12	$90 \times 10 \times 5$	785.6	134.2	97.3	44.1	647	579	675	740.7	44.8	52.7	35	22.9
13	$100 \times 10 \times 5$	897.7	229.8	208.9	190.1	1245	733	1005	648.4	40	52	16	21.1
14	$100 \times 10 \times 6$	912.6	232.6	190.3	195.5	176	631	1376	819.2	32	50	38	8.1
15	$100 \times 20 \times 6$	961.0	86.3	65.9	58.4	258	746	649	822.2	24.3	35.3	13.2	18.4
16	$150 \times 30 \times 6$	2297.4	144.3	88.95	62.6	1059	2611	2655	3496.7	32.2	50.3	38.7	8.2
17	$100 \times 10 \times 11$	629.7	172.5	130	125.7	716	2744	2631	2097.3	58	34.8	20.2	7.1
18	$150 \times 10 \times 11$	1059.4	259.0	195.8	134.9	3635	8810	9850	7882.7	81.3	43.7	30.7	11.5
19	$150 \times 40 \times 11$	1128.4	682.5	469.95	304.4	2316	5573	10681	8578	43	55	30.3	12.7
20	$200 \times 68 \times 15$	1491.4	625.7	527	355.7	4694	7362	8663	9486.3	94	77	35	19.2

86.83%, 43.49%, and 26.32%, respectively, on average. However, the computational time of HDE is, on average, higher by 57.54% and 13.55% than DE and SDE, respectively, and lower by 9.37% than SDEC. In terms of the standard deviation of total tardiness, HDE is, on average, 76.9%, 76.9%, and 52.93% smaller than DE, SDE, and SDEC, respectively. We can, therefore, conclude that HDE obtains the best and most consistent objective values, although it takes relatively more time to do so.

5.4. Experiments on industrial data sets

The industrial data sets consist of 3520 jobs from 20 typical workdays in the textile factory being studied. Their current practice is to generate solutions manually based on the First-In, First-Out or Earliest Due Date rule. Characteristics

Table 8. Computational results on industrial data sets.

No.	Problem scale $n \times m \times c$	Industrial solutions	Average objective value				Standard deviation			
			DE	SDE	SDEC	HDE	DE	SDE	SDEC	HDE
1	$214 \times 68 \times 15$	861.4	752	290.1	135	135	32.2	26.9	2.4	0.6
2	$223 \times 68 \times 15$	963.3	934.6	332.3	251	251	6.7	11.9	0	0
3	$184 \times 68 \times 15$	617.2	593.6	239.7	205.5	205.5	13.5	16	7	1.8
4	$153 \times 68 \times 15$	285.7	264.1	108.4	28	28	14.5	11.6	0	0
5	$162 \times 68 \times 15$	364.6	309.5	75.4	41.7	41	14.6	20.1	14.6	1.1
6	$224 \times 68 \times 15$	992.6	984.9	396.7	330.1	224	30.3	16.3	13	9
7	$202 \times 68 \times 15$	941.9	750.9	319.5	210.5	204.5	29.3	30.7	0	0
8	$197 \times 68 \times 15$	810.7	700.5	346.5	193	165	19.9	28.6	0	0
9	$179 \times 68 \times 15$	745.8	590.5	246.8	155.4	154.5	17.3	48.9	0.3	0
10	$114 \times 68 \times 15$	316.8	70.45	11.4	0	0	4.9	7.6	0	0
11	$129 \times 68 \times 15$	360.1	113.1	20.1	1	1	9.8	8	0.6	0
12	$186 \times 68 \times 15$	1038	546.6	197.9	149.5	149.5	27	35	0.4	0.2
13	$196 \times 68 \times 15$	924.5	585	261.0	115	107.5	32.2	41.2	22.1	0.3
14	$203 \times 68 \times 15$	1002.5	635.5	286.7	124.8	123	26.7	32.8	0.6	0
15	$209 \times 68 \times 15$	875.2	726.4	381.6	226	198.5	15.4	33.9	0	0
16	$195 \times 68 \times 15$	625.9	602.3	262.2	169	169	33.2	55.4	21	7
17	$153 \times 68 \times 15$	321.1	275.4	78.9	33.5	33.5	8.2	20.5	0.3	0.1
18	$90 \times 68 \times 15$	344.9	13.2	0.0	0	0	4.5	0	0	0
19	$124 \times 68 \times 15$	364	88.3	14.6	0	0	6	9.9	0	0
20	$183 \times 68 \times 15$	826.7	818.2	434.4	285	285	10.6	22.3	0	0

of the industrial samples can be found in Section 5.1.1. According to feedback from the company that runs the factory, the target due date of a job is closely related to colour family l_j . l_1 and l_2 are the most common colours produced by the company, whose target due dates are 8 h, and the target due dates for the remaining colours are 24 h. However, the company often fails to reach the target due dates, and the possibility of delay is extremely high. For example, jobs originally intended for delivery within 8 h would take 8–24 h, and jobs originally intended for delivery within 24 h generally take 24–80 h.

For this series of industrial sample tests, our proposed HDE algorithm is compared to real industrial solutions and the three DE variants, and the results are shown in Table 8. Compared to the real industrial solutions, DE, SDE, and SDEC, the proposed HDE algorithm reduces the total tardiness by 81.77%, 76.09%, 42.48% and 6.72%, respectively, on average. The standard deviation of the total tardiness derived by HDE, is, on average, 96.18%, 95.56%, and 87.82% smaller than DE, SDE, and SDEC, respectively. Therefore, HDE is the most effective and reliable algorithm here. For data instances with $n < 130$, by arranging these jobs on 68 machines, results without any tardiness can be obtained using SDEC and HDE. This is because, in industrial practice, jobs that belong to the same colour family have the same processing time and due date. Algorithm 2, which was developed to minimise tardiness according to the relationship between the processing time and due date, therefore has little influence.

6. Conclusion and future work

In this paper, a parallel machine dyeing scheduling problem with different colour families, sequence-dependent setup times and machine eligibility restriction was proposed. An integer programming model was formulated to minimise the total tardiness and tested on up to 16 synthetically generated small-scale data sets using the CVX solver. Since this problem is strongly NP-hard, CVX was applied to solve only the small-sized instances and an effective HDE algorithm was developed for large-scale problem instances. In the proposed algorithm, a special encoding and decoding scheme, as well as chaos theory, were embedded with DE, such that parameters F and CR can be updated adaptively. Moreover, two local search algorithms inspired by two dominance properties were developed to speed up convergence and enhance HDE's exploitation capability. With these two local search algorithms, the sequences of all jobs can be determined beforehand, which means that the decision of the whole problem only involves the assignment of jobs among the machines, thereby speeding up convergence and reducing the computational time. To verify the efficacy of the proposed HDE algorithm, 20 synthetically generated large-scale data sets and 20 industrial data sets were tested. Experimental results showed that the proposed HDE is able to obtain better schedules with smaller objective values for both the synthetic and industrial data sets.

In our future work, more practical considerations, including machine maintenance, failure and re-dyeing, will be considered. Exact algorithms or polynomial-time approximation schemes may also be investigated under some special conditions of this problem.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This research is supported by the National Science Foundation of China (grant number 71801051) and China Postdoctoral Science Foundation (grant number 2018M640396).

ORCID

Debiao Li  <http://orcid.org/0000-0003-4077-3107>

Raymond Chiong  <http://orcid.org/0000-0002-8285-1903>

References

- Afzalirad, Mojtaba, and Javad Rezaeian. 2016. "Resource-Constrained Unrelated Parallel Machine Scheduling Problem with Sequence Dependent Setup Times, Precedence Constraints and Machine Eligibility Restrictions." *Computers and Industrial Engineering* 98: 40–52.
- Anderson, Bradley E., James D. Blocher, Kurt M. Bretthauer, and Munirpallam A. Venkataramanan. 2013. "An Efficient Network-Based Formulation for Sequence Dependent Setup Scheduling on Parallel Identical Machines." *Mathematical and Computer Modelling* 57 (3): 483–493.
- Banitalebi, Akbar, Mohd Ismail Abd Aziz, and Zainal Abdul Aziz. 2016. "A Self-Adaptive Binary Differential Evolution Algorithm for Large Scale Binary Optimization Problems." *Information Sciences* 367–368: 487–511.
- Berkhout, Joost, Eric Pauwels, Rob van der Mei, Jan Stolze, and Siem Broersen. 2020. "Short-Term Production Scheduling with Non-Triangular Sequence-Dependent Setup Times and Shifting Production Bottlenecks." *International Journal of Production Research* 6: 1–25.
- Blum, Christian, Raymond Chiong, Maurice Clerc, Kenneth De Jong, Zbigniew Michalewicz, Ferrante Neri, and Thomas Weise. 2012. "Evolutionary Optimization." In *Variants of Evolutionary Algorithms for Real-World Applications* (pp. 1–29). Berlin Heidelberg: Springer.
- Chen, W. J. 2009. "Scheduling with Dependent Setups and Maintenance in a Textile Company." *Computers and Industrial Engineering* 57 (3): 867–873.
- Hamzadayi, Alper, and Gokalp Yildiz. 2017. "Modeling and Solving Static m Identical Parallel Machines Scheduling Problem with a Common Server and Sequence Dependent Setup Times." *Computers and Industrial Engineering* 106: 287–298.
- Hsu, Hsi-Mei, Yai Hsiung, Ying-Zhi Chen, and Muh-Cherng Wu. 2009. "A GA Methodology for the Scheduling of Yarn-Dyed Textile Production." *Expert Systems with Applications* 36 (10): 12095–12103.
- Huang, Simin, Linning Cai, and Xiaoyue Zhang. 2010. "Parallel Dedicated Machine Scheduling Problem with Sequence-Dependent Setups and a Single Server." *Computers & Industrial Engineering* 58 (1): 165–174.
- Hung, Yi-Feng, Jian-Song Bao, and Yu-En Cheng. 2017. "Minimizing Earliness and Tardiness Costs in Scheduling Jobs with Time Windows." *Computers and Industrial Engineering* 113: 871–890.
- Huynh, Nhat-To, and Chen-Fu Chien. 2018. "A Hybrid Multi-subpopulation Genetic Algorithm for Textile Batch Dyeing Scheduling and An Empirical Study." *Computers and Industrial Engineering* 125: 615–627.
- Jia, Zhao Hong, Kai Li, and Y. T. Leung. 2015. "Effective Heuristic for Makespan Minimization in Parallel Batch Machines with Non-Identical Capacities." *International Journal of Production Economics* 169: 1–10.
- Jiang, Weili, Zengwei Yuan, Jun Bi, and Li Sun. 2010. "Conserving Water by Optimizing Production Schedules in the Dyeing Industry." *Journal of Cleaner Production* 18 (16–17): 1696–1702.
- José, Elias C. Arroyo, Ricardo Gonçalves Tavares, and Joseph Y. T. Leung. 2019. "An Iterated Greedy Algorithm for Total Flow Time Minimization in Unrelated Parallel Batch Machines with Unequal Job Release Times." *Engineering Applications of Artificial Intelligence* 77: 239–254.
- Kim, Jae-Gon, Seokwoo Song, and BongJoo Jeong. 2020. "Minimising Total Tardiness for the Identical Parallel Machine Scheduling Problem with Splitting Jobs and Sequence-Dependent Setup Times." *International Journal of Production Research* 58 (6): 1628–1643.
- Leung, Joseph, Laurie Kelly, and James H. Anderson. 2004. *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. New York: CRC Press, Inc.

- Lu, Youlin, Jianzhong Zhou, Hui Qin, Ying Wang, and Yongchuan Zhang. 2010. "An Adaptive Chaotic Differential Evolution for the Short-Term Hydrothermal Generation Scheduling Problem." *Energy Conversion and Management* 51 (7): 1481–1490.
- Myszkowski, Paweł B., Łukasz P. Olech, Maciej Laszczyk, and Marek E. Skowroński. 2018. "Hybrid Differential Evolution and Greedy Algorithm (DEGR) for Solving Multi-Skill Resource-Constrained Project Scheduling Problem." *Applied Soft Computing* 62: 1–14.
- Naderi-Beni, Mahdi, Ehsan Ghobadian, Sadoullah Ebrahimnejad, and Reza Tavakkoli-Moghaddam. 2014. "Fuzzy Bi-Objective Formulation for a Parallel Machine Scheduling Problem with Machine Eligibility Restrictions and Sequence-Dependent Setup Times." *International Journal of Production Research* 52 (19): 5799–5822.
- Paula, Mateus Rocha, Geraldo Robson Mateus, and Martínez Ravetti. 2010. "A Non-Delayed Relax-and-Cut Algorithm for Scheduling Problems with Parallel Machines, Due Dates and Sequence-Dependent Setup Times." *Computers and Operations Research* 37 (5): 938–949.
- Qian, Bin, Ling Wang, De-Xian Huang, and Xiong Wang. 2006. "Scheduling Multi-objective Job Shops Using a Memetic Algorithm Based on Differential Evolution." *The International Journal of Advanced Manufacturing Technology* 35: 1014–1027.
- Qian, Bin, Ling Wang, De Xian Huang, and Xiong Wang. 2009. "Multi-objective No-Wait Flow-Shop Scheduling with a Memetic Algorithm Based on Differential Evolution." *Soft Computing* 13 (8–9): 847–869.
- Stinson, J. P., and A. W. Smith. 1982. "A Heuristic Programming for Sequencing the Static Flowshop." *International Journal of Production Research* 20: 753–764.
- Storn, Rainer, and Kenneth Price. 1997. "Differential Evolution – A Simple and Efficient Heuristic for Global Optimization Over Continuous Spaces." *Journal of Global Optimization* 11 (4): 341–359.
- Wan, Guohua, and Benjamin P.-C. Yen. 2002. "Tabu Search for Single Machine Scheduling with Distinct Due Windows and Weighted Earliness/Tardiness Penalties." *European Journal of Operational Research* 142 (2): 271–281.
- Wang, Hai-Yan, Yan-Wei Zhao, Xin-Li Xu, and Wan-Liang Wang. 2009. "Scheduling Batch and Continuous Process Production Based on An Improved Differential Evolution Algorithm." *System Engineering Theory and Practice* 29 (11): 157–167.
- Wu, Xueqi, and Ada Che. 2019. "A Memetic Differential Evolution Algorithm for Energy-Efficient Parallel Machine Scheduling." *Omega* 82: 155–165.
- Yuan, Yuan, and Hua Xu. 2013. "Flexible Job Shop Scheduling Using Hybrid Differential Evolution Algorithms." *Computers and Industrial Engineering* 65 (2): 246–260.
- Yuan, Xiaohui, Yongchuan Zhang, Liang Wang, and Yanbin Yuan. 2008. "An Enhanced Differential Evolution Algorithm for Daily Optimal Hydro Generation Scheduling." *Computers and Mathematics with Applications* 55 (11): 2458–2468.
- Zhang, Rui, Pei-Chann Chang, Shiji Song, and Cheng Wu. 2017. "A Multi-objective Artificial Bee Colony Algorithm for Parallel Batch-Processing Machine Scheduling in Fabric Dyeing Processes." *Knowledge-Based Systems* 116: 114–129.
- Zhou, Shengchao, Ming Liu, Huaping Chen, and Xueping Li. 2016. "An Effective Discrete Differential Evolution Algorithm for Scheduling Uniform Parallel Batch Processing Machines with Non-Identical Capacities and Arbitrary Job Sizes." *International Journal of Production Economics* 179: 1–11.
- Zhou, Lu, Kun Xu, Xun Cheng, Yangyu Xu, and Qibo Jia. 2017. "Study on Optimizing Production Scheduling for Water-Saving in Textile Dyeing Industry." *Journal of Cleaner Production* 141: 721–727.